

Data Structures (CS2103)

Introduction

Prepared By – Ashish K Layek
CST, IEST Shibpur

Motivation

- ❑ Computers do store, retrieve, and process a large amount of data.
 - ❑ If the data is stored in well organized way on storage media and in computer's memory, it can be accessed quickly for processing and the user is provided fast response.
-

What is Data Structure ?

In computer science, a data structure is a data organization, management, and storage format that is usually chosen for efficient access to data.

More precisely, a data structure is a collection of data values, the relationships among them, and the functions or operations that can be applied to the data, i.e., it is an algebraic structure about data.

Wikipedia

Data Structures

Definitions

Data Structure is a way of collecting and organising data in such a way that we can perform operations on these data in an effective way.

In simple language, Data Structures are structures programmed to store ordered data, so that various operations can be performed on it easily.

- ❑ Ways to organize and store data
 - Data Storages
 - ❑ Ways to access and manipulate the stored data
 - Methods to access storages
-

Function

- ❑ Data structure is a scheme for data organization.
 - ❑ The functional definition of a data structure should be independent of its implementation.
 - ❑ The functional definition of a data structure is known as ADT (Abstract Data Type) which is independent of implementation.
 - ❑ The implementation part is left to the developers who decide which technology better suits to their project needs.
 - ❑ For example, a stack ADT is a structure which supports operations such as push() and pop().
 - A stack can be implemented in a number of ways, for example using an array or using a linked list.
-

Algorithm

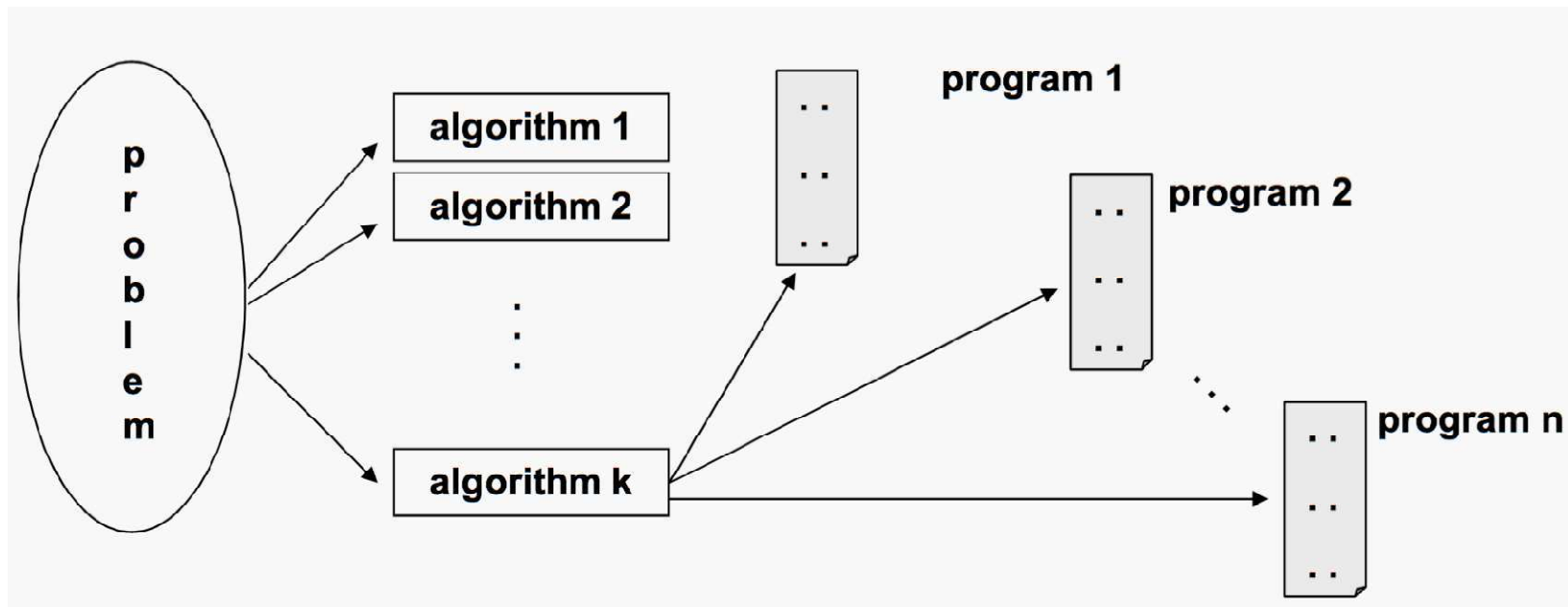
- ❑ Along with data structures, in real life, problem solving is done with help of Algorithms.
 - ❑ An algorithm is a step by step process to solve a problem.
 - **Finiteness**: The algorithm must always terminate after a finite number of steps
 - **Definiteness**: Each step must be precisely defined
 - **Input**: An algorithm has zero or more inputs, taken from a specified set of objects
 - **Output**: An algorithm has one or more outputs, which have a specified relation to the inputs
 - **Effectiveness**: its operations must be basic enough to be able to be done exactly and in a finite length of time
 - ❑ In programming, algorithms are implemented in form of methods or functions or routines.
-

Efficient Algorithm

- ❑ To get a problem solved we not only want algorithm but also an efficient algorithm.
 - ❑ One criteria of efficiency is time taken by the algorithm, another could be the **memory/space** it takes at run time.
 - ❑ We can have more than one algorithm for the same problem.
 - ❑ The best algorithm is the one which has a fine balance between time taken and memory consumption.
 - ❑ Sadly, Best rarely exists.
 - ❑ We generally give more priority to the time taken by the algorithm rather than the memory it consumes.
-

Problems vs Algorithms vs Programs

- ❑ For each problem or class of problems, there may be many different algorithms.
- ❑ For each algorithm, there may be many different implementations (programs).



Categories of Data Structure

❑ Linear Data Structure

- Elements combine to form any specific order
 - ✓ Arrays
 - ✓ Queues
 - ✓ Stacks
 - ✓ Linked lists

❑ Non-linear Data Structure

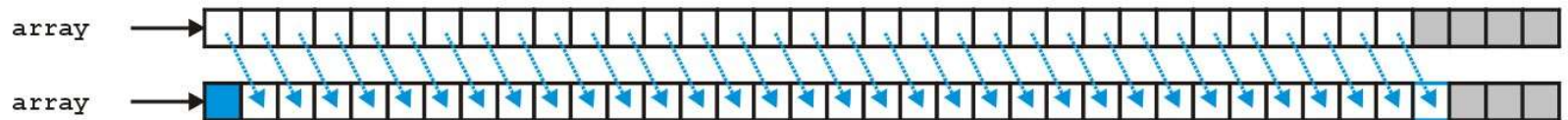
- Hierarchical relationship among various elements
 - ✓ Graphs
 - ✓ Family of trees
 - ✓ Hash Table
 - ✓ Heap
-

Array

- ❑ Arrays are by far the most common data structure used to store data.
- ❑ Arrays are generally statically implemented data structures by some programming languages.

Disadvantages:

- **Inserting / Deleting** an item to/from an array
- The time taken by insert operation depends on how big the array is, and at which position the item is being inserted.



Advantages:

- **Searching:**
 - ✓ If the array is unsorted then search operation is also proved costly and takes $O(N)$ time in worst case, where N is size of the array. If the array is sorted then search performance is improved magically and takes $O(\lg N)$ time in worst case.

Array: Contiguous Allocation

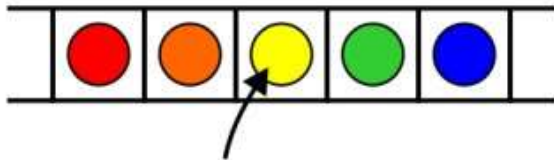
- ❑ An array stores n objects in a single contiguous space of memory
- ❑ Unfortunately, if more memory is required, a request for new memory usually requires copying all information into the new memory
 - In general, you cannot request for the operating system to allocate to you the next n memory locations



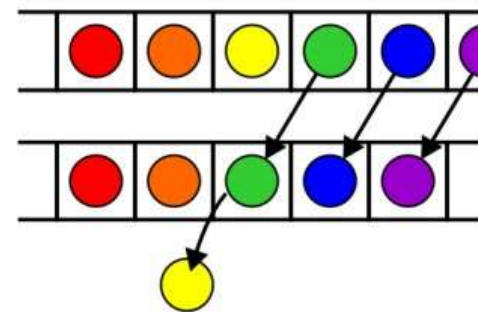
Array: Operations

- Operations at the k th entry of the array include:

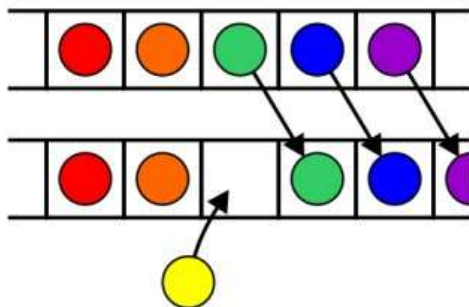
Access to the object



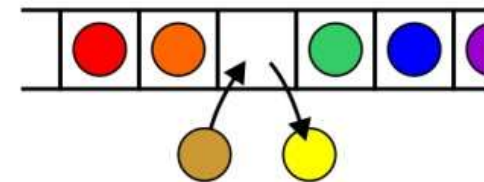
Erasing an object



Insertion of a new object

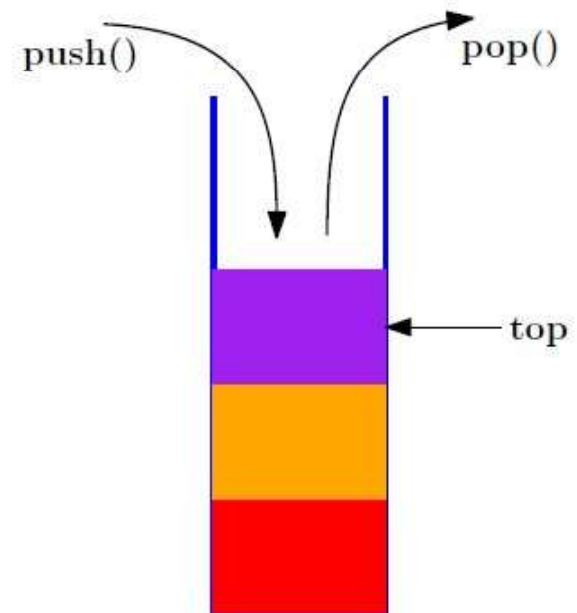
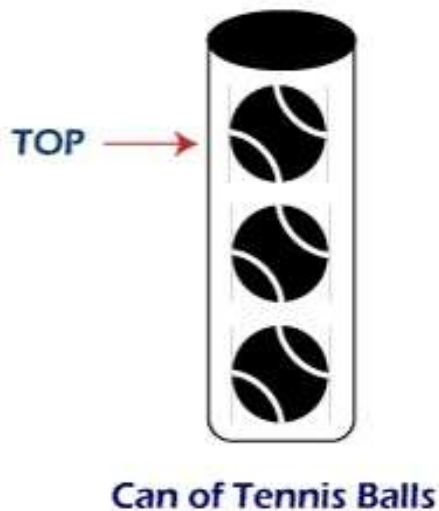


Replacement of the object



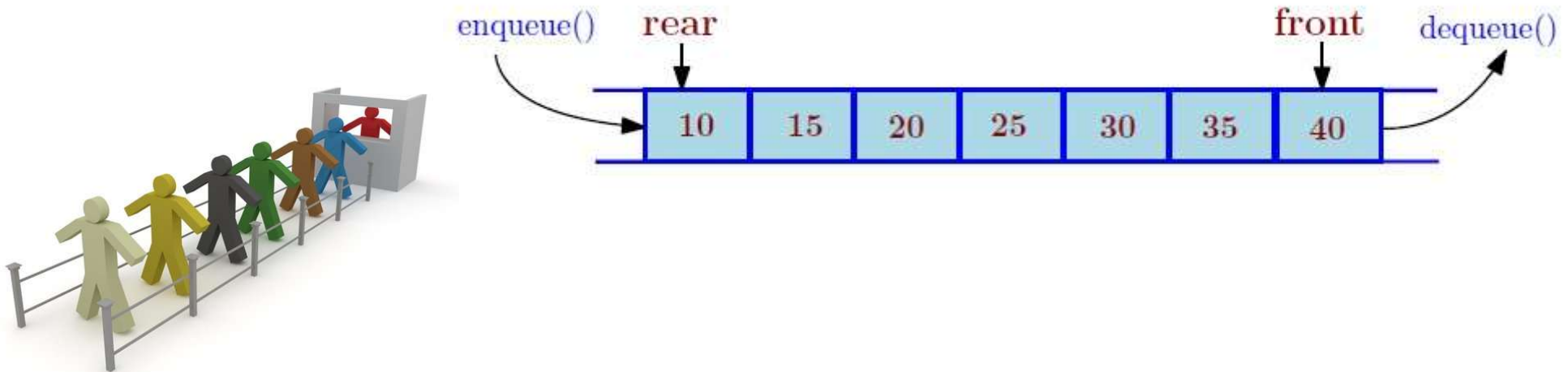
Stack

- ❑ A container of objects that are inserted and removed according to the **last-in-first-out (LIFO)** principle.
- ❑ Only the last (**the most recently inserted**) object can be removed.



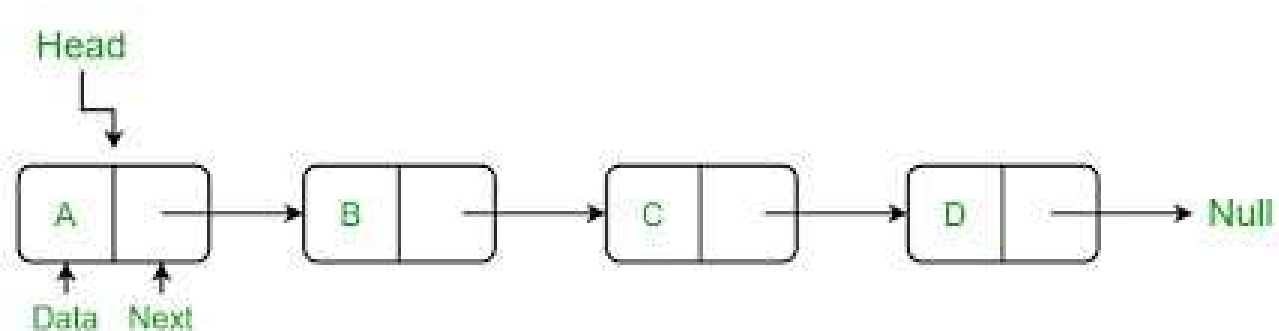
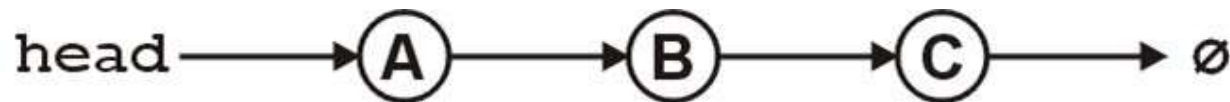
Queue

- ❑ Differs from a stack in that its insertion and removal follows the **first-in-first-out (FIFO)** principle.
- ❑ The element which has been **in the queue the longest** may be removed.



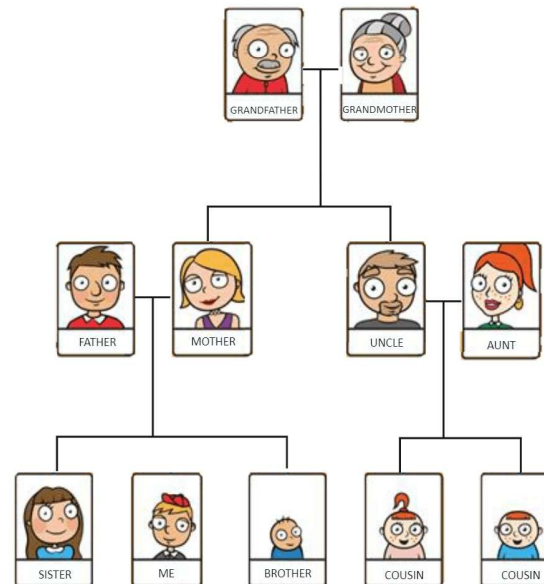
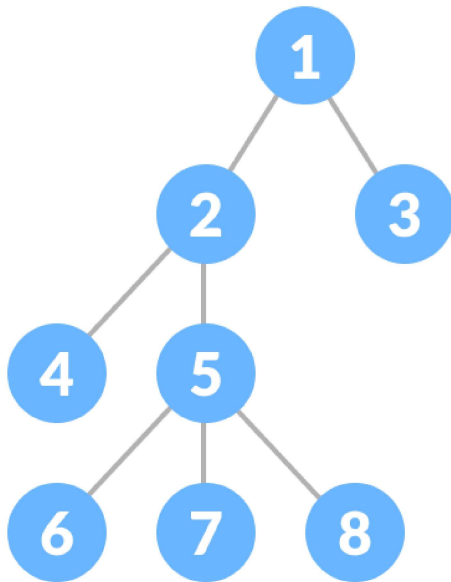
Linked List

- ❑ A collection of linearly arranged element (a linear order).
- ❑ Provides methods for accessing, inserting, and removing arbitrary elements.
- ❑ Notion of position, before and after.



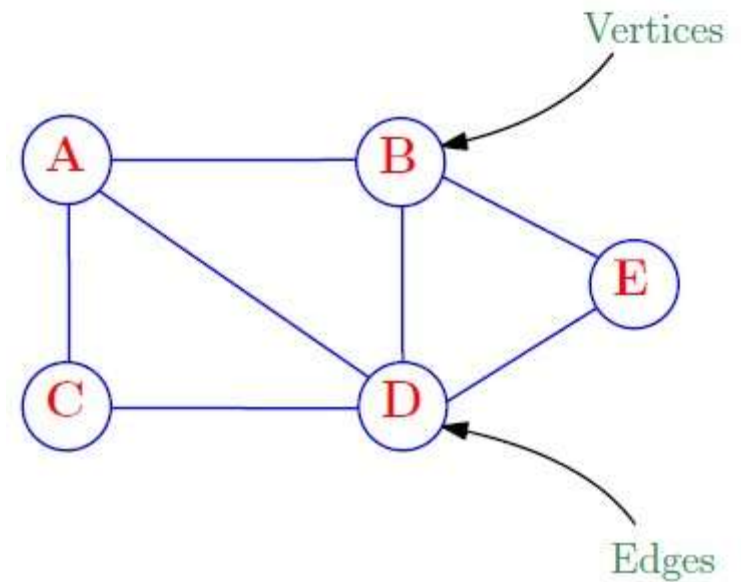
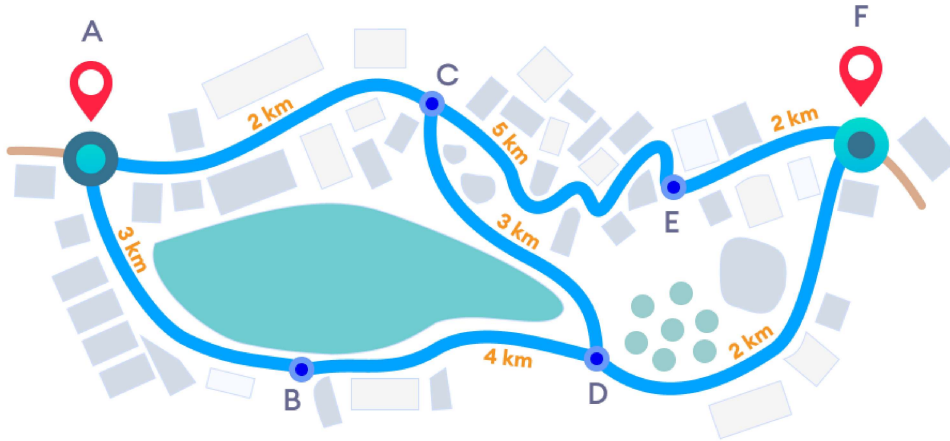
Tree

- ❑ A collection of objects arranged in a **hierarchical fashion**.
- ❑ E.g., organization of a corporation, a table of content, dos/unix file systems, family tree.
- ❑ Notion of **parents and children, root and leaves**.



Graphs

- ❑ Representing a way of **connections or relationships between pairs of objects**.
- ❑ Not only physical connectivity, but also **logical relationship**.



And many more ...

Abstract Data Types and Data Structures

Often, these terms are used as synonyms. But its better to think of them this way:

- ❑ An Abstract Data Type (ADT) represents a particular set of behaviours.
 - You can formally define (i.e., using mathematical logic) what an ADT is/does.
 - e.g., a Stack is a list implements a LIFO policy on additions/deletions.

 - ❑ A data structure is more concrete. Typically, it is a technique or strategy for implementing an ADT.
 - Use a linked list or an array to implement a stack class.
-

Abstract Data Types and Data Structures

- ❑ Some common ADTs that all trained programmers know about:
 - stack, queue, priority queue, dictionary, sequence, set.

 - ❑ Some common data structures used to implement those ADTs:
 - array, linked list, hash table (open, closed, circular hashing)
 - trees (binary search trees, heaps, AVL trees, 2-3 trees, tries, red/black trees, B-trees)
-

Books

- ✓ Data Structures using C and C++
by Y. Langsam, M. Augenstein And A. M. Tenenbaum
 - ✓ Data Structures and Program Design in C
by R. Kruse and B. Leung
 - ✓ Fundamentals of Data Structures in C
by E. Horowitz, S. Sahni, S. Anderson-Freed
 - ✓ Data Structures – A Pseudocode Approach with C++
by R. F. Gillbert and B. A. Forouzan
-